Comerciales Mely: desarrollo móvil con React Native y diseño UX/UI

Universidad Don Bosco, Facultad de Ingeniería

Diseño y Programación de Software Multiplataforma - DPS941

Ciclo II-2026 · Equipo R.E.F.E.S

Etapa 3: Avance 1 (Foro de Discusión 1)

19 de octubre de 2026

Cerón Contreras, Edwin Wilfredo - CC060761

Moreno Hernández, Jorge Steven - MH222714

Mulato Vasquez, Raúl Adalberto - MV222715

López Torres, Fernando Raúl - LT172001

Rivera Cubias, Edwin Vladimir - RC232947

Docente: Alexander Alberto Siguenza Campos

Introducción

Comerciales Mely necesita consultar existencias, registrar movimientos de bodega y procesar ventas desde un dispositivo móvil. La aplicación web defendida en la etapa 2 constituye la referencia funcional: utiliza tres perfiles de trabajo y relaciona inventario, movimientos y comprobantes. En la versión móvil se conserva esa relación para que una venta tenga consecuencias verificables en las existencias y en el historial, en lugar de limitarse a mostrar formularios independientes.

Este informe documenta el primer avance móvil y distingue su alcance de las entregas posteriores. La ruta de aprendizaje establece para el Foro 1 autenticación, navegación funcional, validación, coherencia con los mockups y un documento sobre React Native y Figma (Universidad Don Bosco, 2026a). La aplicación implementa una base local con SQLite; la autenticación de Firebase, Google y la base compartida con la web pertenecen al siguiente avance y a la entrega final. No se presentan estos últimos requisitos como cumplidos.

Objetivos

El objetivo general es construir una aplicación móvil de Comerciales Mely con navegación y módulos de negocio operativos, aplicando los contenidos de las unidades 1, 2 y 3. Los objetivos específicos son separar presentación, lógica y datos; validar los formularios; diferenciar las acciones por perfil; conservar las operaciones al cerrar la app; y proporcionar un procedimiento reproducible de ejecución y evaluación.

1. De la aplicación web a componentes nativos

React Native ofrece componentes como View, Text, TextInput, Pressable, Image y Modal para construir interfaces móviles. FlatList resulta adecuada para catálogos extensos porque organiza el renderizado de las filas visibles (Meta Platforms, s. f.). En Comerciales Mely se utiliza para consultar productos y seleccionar artículos en ventas o bodega. Los formularios se presentan con contenedores desplazables y tratamiento del teclado; no se incrusta la web dentro de un navegador.

Los contenidos de JavaScript de la unidad 1 se aplican en funciones puras, arreglos, objetos, condiciones y validación de cantidades. Las unidades 2 y 3 sustentan componentes, props, estado, eventos, StyleSheet, navegación y almacenamiento móvil (Universidad Don Bosco, 2026b, 2026c, 2026d). Los componentes funcionales permiten compartir Field, Button, Card, Screen y los mensajes de error entre pantallas. Cada componente recibe propiedades explícitas y el estado cambia mediante hooks.

El proyecto utiliza Expo SDK 58, React Native 0.88 y React 19.3.0. Estas versiones exigen revisar la documentación correspondiente y no copiar sin adaptación los números de versión de ejemplos antiguos. La tabla de compatibilidad de Expo establece Node.js 22.13 o superior para este SDK (Expo, s. f.-a). El equipo dispone de Node.js 24; el script Android selecciona el JDK incluido en Android Studio para no depender del Java 27 global.

2. Proceso UX/UI y relación con Figma

La tarea principal guía la jerarquía visual: el usuario identifica su situación en Inicio, encuentra un producto en Inventario y registra la operación en Bodega o Ventas. Las acciones de eliminación se convierten en archivo lógico para conservar las referencias históricas. Se solicita confirmación antes de emitir una venta o cerrar la sesión. Los estados vacíos explican cómo continuar y los errores permanecen junto a la acción que los originó.

Figma Auto Layout permite organizar componentes por dirección, separación, relleno y alineación; el contenido se redistribuye cuando cambia su tamaño (Figma, s. f.). La correspondencia propuesta en React Native utiliza flexDirection, gap, padding y alignItems. Para comprobarla se deben contrastar las pantallas reales con los frames originales y documentar las diferencias de tipografía, color, espacio y flujo.

El prototipo original fue inspeccionado en Figma: login-movil (4:1226), inventario-movil (4:1275), detalle-producto-movil (4:1363), registro-movimiento-movil (4:1406) y alertas-stock-movil (4:1467), del archivo 11GPTpTj3RrHusfOA9qqOr. La implementación utiliza el acento naranja #D97706, texto #111827, fondo #F3F4F6, bordes #E5E7EB y tipografía Inter. Las imágenes e iconos se descargan como recursos locales. La barra de estado y las áreas seguras se delegan al sistema operativo, sin dibujar una hora o batería ficticias.

Los controles táctiles tienen una altura mínima de 48 unidades, etiquetas de accesibilidad y campos identificados por su propósito. El color se acompaña de texto como “Stock bajo” o “Disponible”. Las áreas seguras reducen interferencias con los bordes del dispositivo y las pestañas mantienen visibles las rutas principales. La visualización de reportes incluye valores y fechas además de barras para no depender únicamente de la forma gráfica.

3. Arquitectura, navegación y manejo del estado

La presentación está en src/app y src/components; la lógica de negocio está en src/domain/business.mjs; el acceso a SQLite y al almacén seguro se concentra en src/services. AppContext conecta la sesión y los datos con las pantallas. La separación permite probar las reglas sin montar una interfaz y reemplazar posteriormente la persistencia sin reescribir cada formulario.

Expo Router organiza las rutas como archivos y utiliza la infraestructura de React Navigation. Las rutas públicas son login y registro; el grupo autenticado contiene Inventario,

Movimientos, Alertas, Perfil, Ficha Técnica, Resumen, Ventas y Reportes. Las cuatro primeras forman la barra principal del prototipo; las herramientas adicionales se abren desde Perfil. En Expo Router 58, href nulo se convierte internamente en una ruta oculta con redirección. Para conservar accesibles las herramientas secundarias se oculta únicamente su botón con tabBarButton y tabBarItemStyle, sin marcar la ruta como hidden. La prueba del APK permitió detectar y corregir esta diferencia respecto de ejemplos de versiones anteriores. Stack.Protected condiciona el acceso a la sesión activa (Expo, s. f.-b). Se declara redirectTo de forma explícita: el grupo autenticado vuelve a /login al revocarse la sesión y las rutas públicas vuelven a /inventario al ingresar. El recorrido en Android valida también esta transición, porque persistir o revocar el token no sustituye comprobar la pantalla de destino. Las pantallas de ventas y reportes revisan adicionalmente el perfil, y la capa de datos valida cada operación antes de escribir. Ocultar una pestaña por sí solo no se considera autorización suficiente.

El administrador puede gestionar productos, archivar, registrar movimientos y vender. El empleado consulta y gestiona inventario y bodega, pero no emite ventas. El facturador consulta productos y movimientos, emite comprobantes y revisa reportes, pero no altera directamente las existencias. El registro público permite únicamente empleado o facturador. Son permisos de un prototipo local: una aplicación distribuida entre dispositivos necesitará autorización impuesta por el backend o por las reglas de Firebase.

El flujo de datos es: evento táctil, validación de entrada, comprobación de sesión y perfil, transición de negocio, transacción SQLite y actualización del Context. El formulario muestra el error si el proceso falla. Las escrituras se serializan y los botones muestran un estado de trabajo para reducir operaciones duplicadas.

4. Persistencia, autenticación y validaciones

SQLite conserva productos, usuarios, sesiones, movimientos y comprobantes al cerrar la aplicación. La API expo-sqlite permite consultas parametrizadas y transacciones exclusivas (Expo, s. f.-c). Este avance almacena el estado de negocio versionado en una fila JSON dentro de SQLite. Es una decisión para el alcance académico local; antes de un catálogo grande o de sincronización multiusuario conviene normalizar tablas, añadir migraciones y trasladar la autoridad de escritura al servidor.

Las contraseñas se almacenan mediante bcrypt con sal aleatoria; el token de sesión se conserva en SecureStore y su validez se comprueba contra el estado local. La sesión dura hasta ocho horas y el cierre elimina su token del almacén seguro y lo revoca en SQLite. SecureStore utiliza mecanismos del sistema operativo para guardar valores sensibles (Expo, s. f.-d). Las cuentas de demostración se inicializan una sola vez y están destinadas exclusivamente a la evaluación local.

Zod valida nombres, usuario, contraseña, perfil, precios, cantidades y motivos de movimiento. El registro exige al menos ocho caracteres y confirmación coincidente. Los precios deben ser como mínimo 0.01; las cantidades de inventario y movimientos deben ser enteras. Se rechazan ventas sin artículos, cantidades nulas o negativas y stock insuficiente. Los mensajes incluyen el campo afectado y las operaciones fallidas no actualizan parcialmente el inventario.

La imagen del producto se selecciona desde la biblioteca o se captura con la cámara. La cámara solicita autorización; si se deniega, la aplicación explica cómo habilitarla. El archivo seleccionado se copia al directorio de documentos de la app para no depender de una ubicación temporal. Los metadatos se conservan junto al producto. El usuario puede cancelar la selección sin perder el formulario.

5. Módulos con lógica de negocio

Inventario permite buscar por nombre o categoría, crear y editar productos, mostrar imágenes y detectar stock igual o inferior al mínimo. Una edición no puede modificar directamente las existencias: debe registrarse un movimiento, con motivo y usuario, para mantener la trazabilidad. El administrador puede archivar productos; el comprobante emitido conserva el nombre y precio originales aunque el catálogo cambie posteriormente.

Bodega registra entradas y salidas con cantidades enteras. Una salida superior a las existencias se rechaza. Cada operación conserva producto, tipo, cantidad, motivo, usuario y fecha. Las ventas generan automáticamente sus movimientos de salida, lo que permite revisar cómo se alcanzó el stock actual.

Facturación agrupa artículos repetidos antes de validar el stock. Los importes se calculan en centavos y el IVA se redondea una vez sobre el subtotal. Por ejemplo, dos artículos de $3.50 y uno de $5.25 suman $12.25; el IVA de demostración del 13 % es $1.59 y el total es $13.84. La tasa se conserva de la propuesta académica y de la web de referencia; no se presenta como asesoramiento tributario.

La venta valida todas sus líneas antes de escribir. Stock, historial y comprobante se guardan en una única transacción. Si falla una línea, no se aplica ninguna salida. Los comprobantes tienen numeración incremental local y pueden compartirse como PDF. Son documentos académicos de demostración y no documentos tributarios electrónicos autorizados.

Reportes calcula ventas por día usando el calendario local del dispositivo, cantidad de comprobantes, alertas de reposición y valor del inventario a precio de venta sin IVA. Este último indicador no representa ganancia: no se conoce el costo de adquisición. La gráfica muestra los últimos siete días y se actualiza con los datos reales guardados en la app.

6. Ejecución Android e iOS

Para reproducir el proyecto se instala Node.js compatible, se ejecuta npm ci en la raíz y luego npm run start para desarrollo. Android requiere el SDK configurado en Android Studio. El archivo scripts/build-android.ps1 selecciona el JDK de Android Studio y genera el APK con el bundle JavaScript incorporado. La firma de demostración permite instalación académica; publicar en una tienda requiere una clave de distribución propia.

El APK de entrega se puede instalar por USB o en un emulador mediante adb install -r. A diferencia de una ejecución dependiente de Metro, el APK con bundle puede abrirse sin mantener una terminal de desarrollo y los módulos locales funcionan sin internet. Los resultados y registros de las comprobaciones deben consultarse en output/evidence.

En Windows no se dispone de Xcode para compilar localmente iOS. EAS ofrece compilación en infraestructura macOS (Expo, s. f.-e). El perfil preview de eas.json está preparado para distribución interna y ios-simulator para un simulador de Mac. Una compilación instalable en iPhone requiere iniciar sesión en Expo, asociar un proyecto y aportar las credenciales y dispositivos autorizados por Apple. El archivo scripts/build-ios.ps1 ejecuta ese flujo. Una exportación del bundle iOS verifica empaquetado JavaScript, pero no equivale a producir ni probar un archivo IPA.

7. Verificación y evidencias

Las pruebas automatizadas del dominio verifican redondeo monetario, actualización de stock, rechazo de artículos duplicados que exceden existencias, reversión ante una línea inválida, permisos por rol, cantidades fraccionarias, registro sin elevación a administrador, archivo de productos y reportes por fecha local. La evidencia reproducible se obtiene mediante npm test. Lint, revisión sintáctica y exportaciones de plataforma complementan la verificación, sin sustituir las pruebas en el dispositivo. La ejecución 39199889132 de GitHub Actions compiló el APK y completó once comprobaciones de interfaz en Android 14: acceso, validación, catálogo, ficha, entrada, venta, reportes, persistencia, dos perfiles y cierre de sesión. Las primeras nueve capturas del anexo y el registro JSON corresponden a esa ejecución real. Una comprobación adicional sobre el mismo APK confirmó registro de una cuenta de empleado, reingreso y cierre; sus tres capturas y registro están incluidos como evidencia adicional.

El recorrido manual recomendado es iniciar sesión con los tres perfiles, registrar una cuenta, buscar un producto, guardar una imagen, crear una entrada, intentar una salida excesiva, emitir una venta y comprobar el descuento en Inventario y el importe en Reportes. Después debe cerrarse y reabrirse la app para revisar persistencia. El video debe mostrar errores controlados y explicar el código responsable, no únicamente recorrer pantallas.

Las capturas incorporadas en el anexo provienen de la app ejecutada y deben leerse junto al registro de pruebas. Los requisitos externos pendientes se enumeran en docs/CRITERIOS.md: enlace del video con explicación del equipo y participación real de cada integrante. No se inventan contribuciones, resultados de dispositivos o enlaces que no existan.

Conclusiones

La versión móvil transforma los módulos de Comerciales Mely en acciones conectadas: vender modifica el stock, genera un movimiento y actualiza los reportes. Separar estas reglas de las pantallas permite verificarlas de forma independiente. Los componentes reutilizables y las áreas seguras favorecen una presentación consistente en tamaños de pantalla diferentes.

La persistencia local permite demostrar los contenidos de las unidades estudiadas y operar sin conexión. Esta solución tiene límites claros: no comparte datos entre dispositivos, no integra todavía Firebase y no demuestra por sí sola una compilación firmada de iOS. Para completar toda la etapa 3 se debe implementar la autenticación por correo y Google, Firestore con reglas seguras, sincronización con la web y pruebas reales en ambas plataformas.

El prototipo se consulta en https://www.figma.com/design/11GPTpTj3RrHusfOA9qqOr/?node-id=4-1226 . Las adaptaciones conservan la captura por usuario para compatibilidad con la web e incorporan alias de correo para las cuentas de demostración. El selector de rol comprueba el perfil almacenado; no concede privilegios. La defensa debe explicar las decisiones, ejecutar casos de error y reconocer el alcance real. La documentación no sustituye la comprensión del equipo ni permite afirmar que se satisfacen evidencias externas pendientes.

Referencias

Expo. (s. f.-a). Expo SDK reference. Recuperado el 18 de octubre de 2026, de https://docs.expo.dev/versions/v58.0.0/

Expo. (s. f.-b). Authentication in Expo Router. Recuperado el 18 de octubre de 2026, de https://docs.expo.dev/router/advanced/authentication/

Expo. (s. f.-c). SQLite. Recuperado el 18 de octubre de 2026, de https://docs.expo.dev/versions/v58.0.0/sdk/sqlite/

Expo. (s. f.-d). SecureStore. Recuperado el 18 de octubre de 2026, de https://docs.expo.dev/versions/v58.0.0/sdk/securestore/

Expo. (s. f.-e). Create your first build. Recuperado el 18 de octubre de 2026, de https://docs.expo.dev/build/setup/

Figma. (s. f.). Guide to auto layout in Figma. Recuperado el 18 de octubre de 2026, de https://help.figma.com/hc/en-us/articles/360040451373-Guide-to-auto-layout-in-Figma

Meta Platforms. (s. f.). Core components and APIs. React Native. Recuperado el 18 de octubre de 2026, de https://reactnative.dev/docs/components-and-apis

Universidad Don Bosco. (2026a). Ruta de aprendizaje: Diseño y programación de software multiplataforma DPS941, ciclo II-2026 [Material de cátedra]. Dirección de Educación a Distancia.

Universidad Don Bosco. (2026b). Unidad 1: Introducción al lenguaje de programación JavaScript [Material de cátedra]. Dirección de Educación a Distancia.

Universidad Don Bosco. (2026c). Unidad 2: Introducción a React, biblioteca de JavaScript y React Native [Material de cátedra]. Dirección de Educación a Distancia.

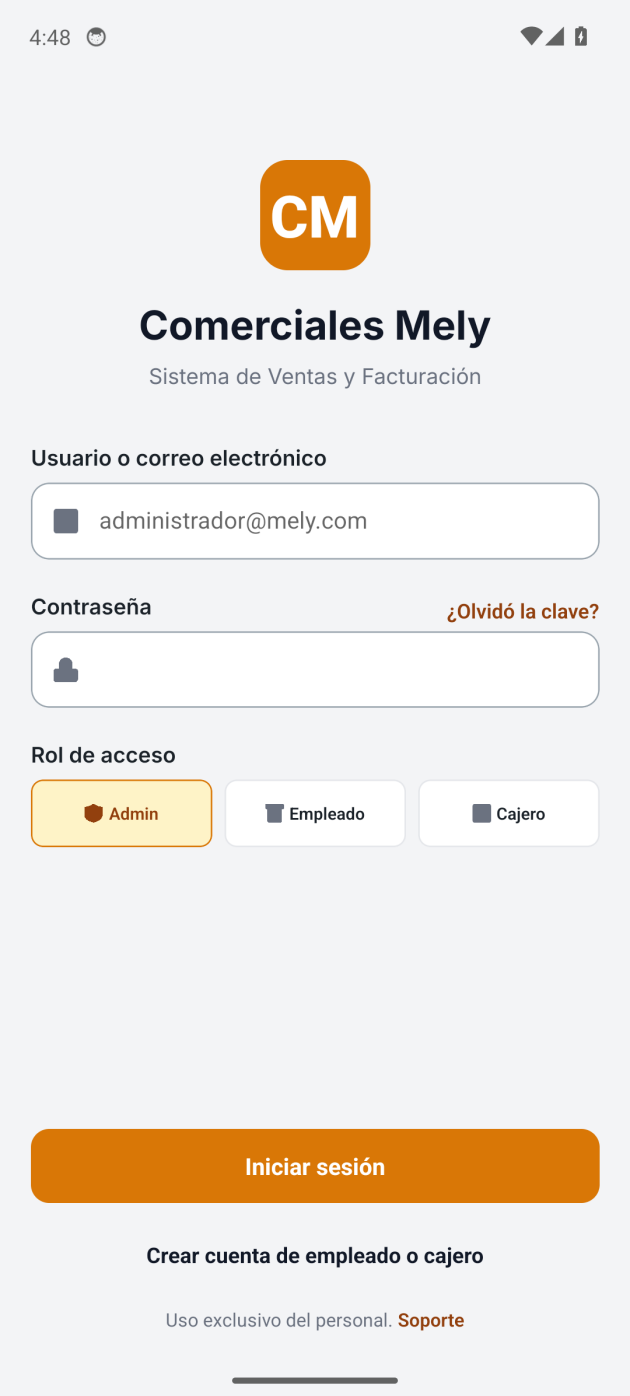
Universidad Don Bosco. (2026d). Unidad 3: Herramientas de diseño y desarrollo en React Native [Material de cátedra]. Dirección de Educación a Distancia.

Equipo R.E.F.E.S. (2026). Comerciales Mely: Sistema de gestión multiplataforma [Código fuente]. GitHub. https://github.com/m4c4c0/comerciales-mely

wilfredo409. (2026). Comerciales Mely: Referencia web [Código fuente]. GitHub. https://github.com/wilfredo409/comerciales-mely-web

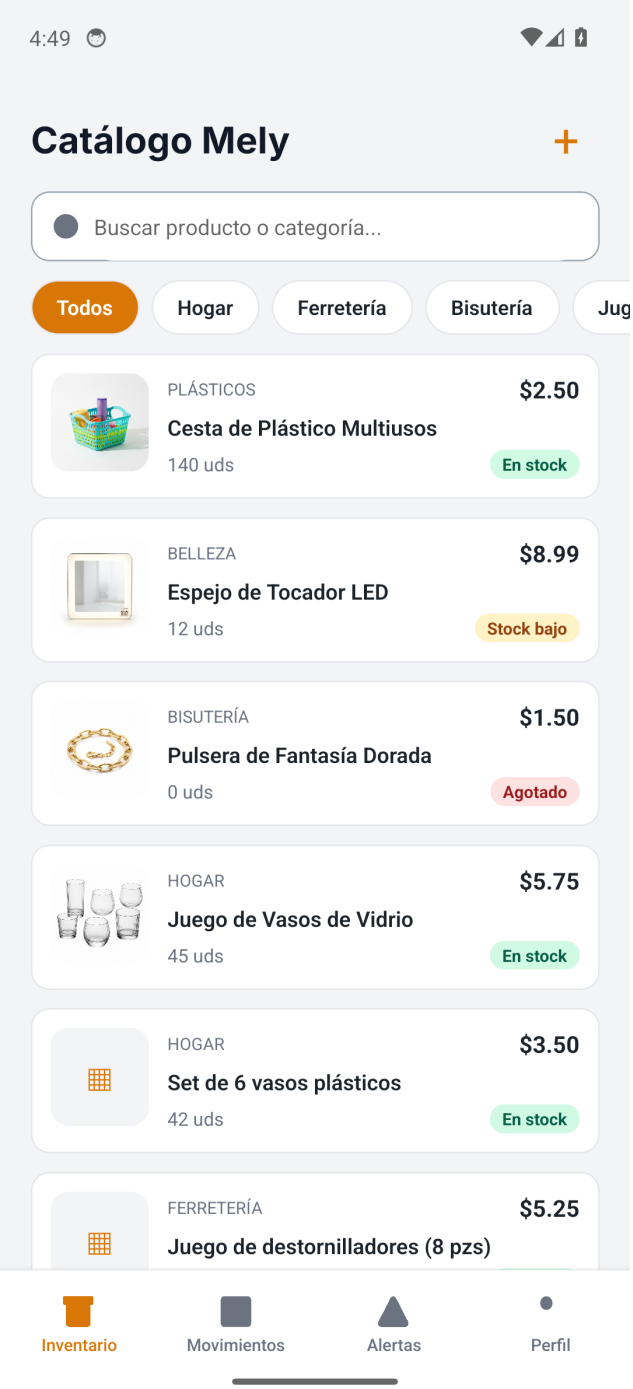
Anexo: evidencias de ejecución Android

Figura 1



Login. Nota. Captura de la aplicación ejecutada en Android. Elaboración propia.

Figura 2



Inventario. Nota. Captura de la aplicación ejecutada en Android. Elaboración propia.

Figura 3



Producto. Nota. Captura de la aplicación ejecutada en Android. Elaboración propia.

Figura 4



Movimientos. Nota. Captura de la aplicación ejecutada en Android. Elaboración propia.

Figura 5



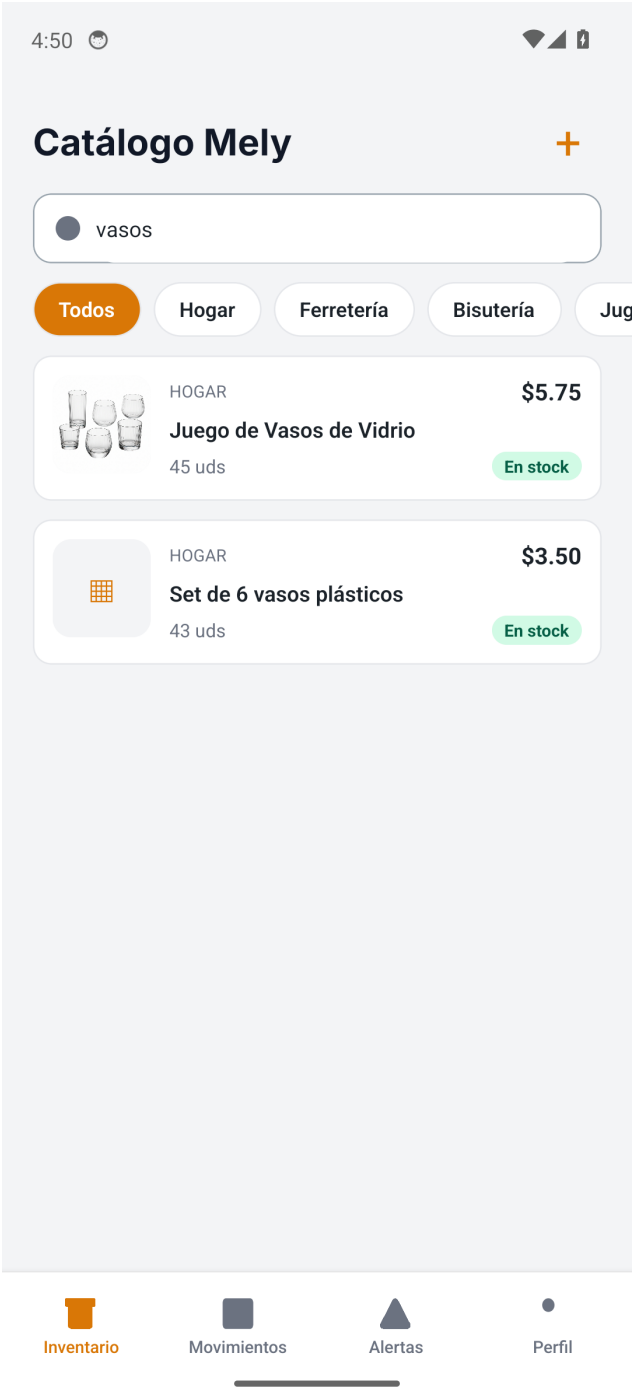
Ventas. Nota. Captura de la aplicación ejecutada en Android. Elaboración propia.

Figura 6



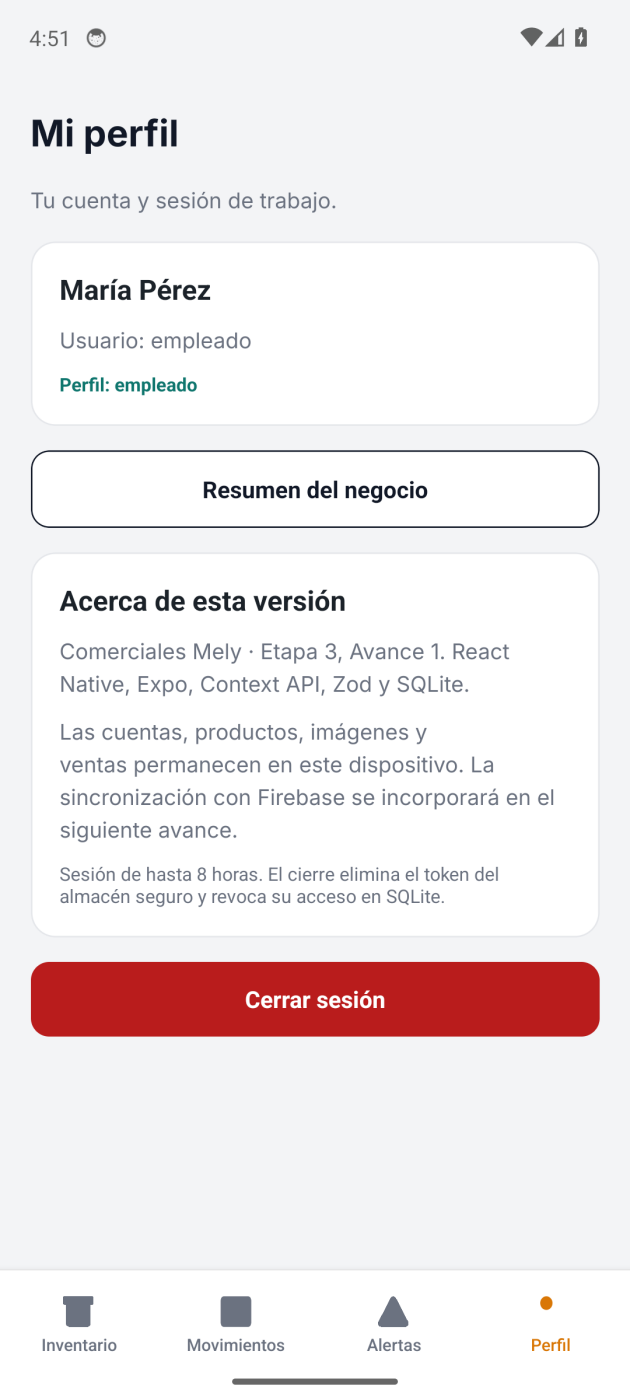
Reportes. Nota. Captura de la aplicación ejecutada en Android. Elaboración propia.

Figura 7



Persistencia. Nota. Captura de la aplicación ejecutada en Android. Elaboración propia.

Figura 8



Permisos empleado. Nota. Captura de la aplicación ejecutada en Android. Elaboración propia.

Figura 9



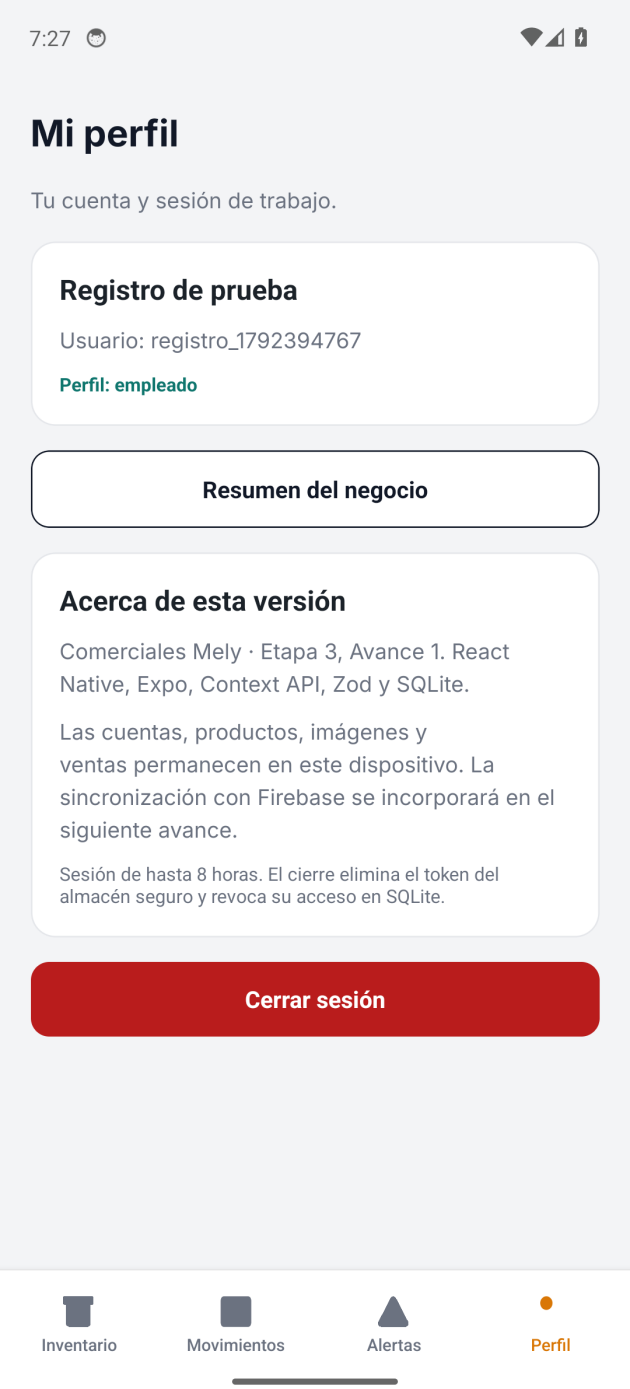
Permisos cajero. Nota. Captura de la aplicación ejecutada en Android. Elaboración propia.

Figura 10



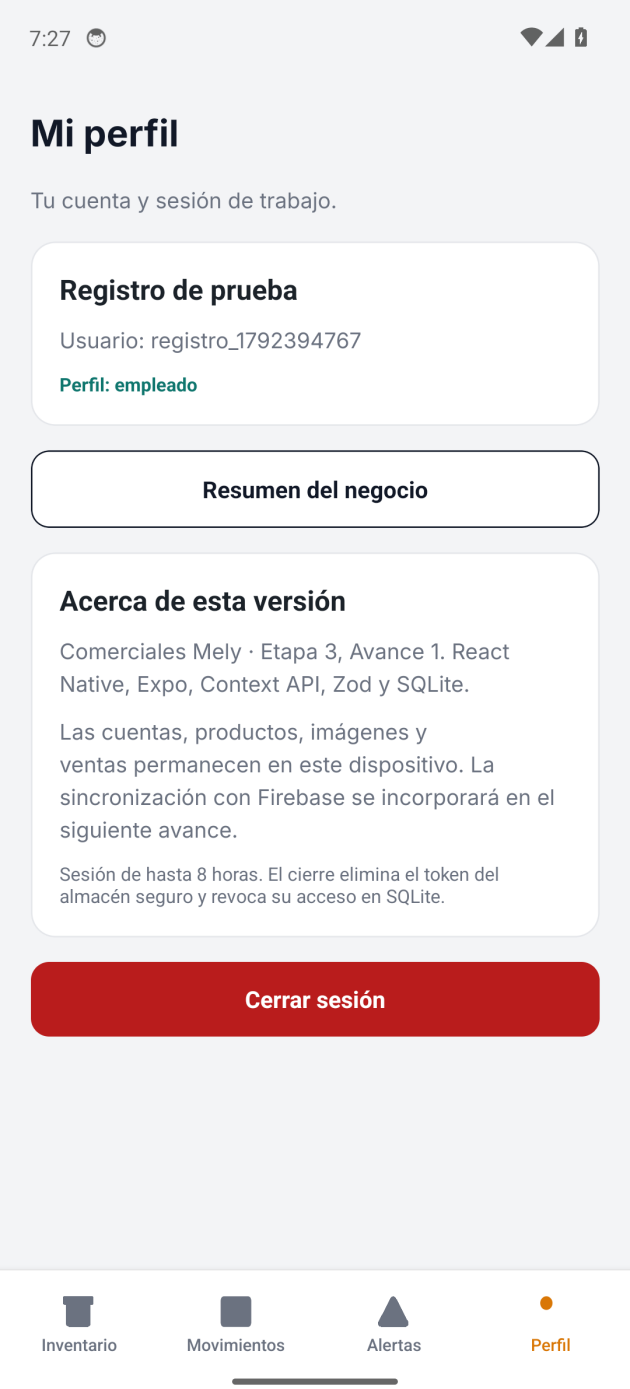
Registro. Nota. Captura de la aplicación ejecutada en Android. Elaboración propia.

Figura 11



Registro creado. Nota. Captura de la aplicación ejecutada en Android. Elaboración propia.

Figura 12



Registro reingreso. Nota. Captura de la aplicación ejecutada en Android. Elaboración propia.